MTB Galaxy

Реализация проекта по исходному плану в формате Word

Актуализировано по состоянию на 23.04.2026

Документ подготовлен на основе исходного Word-плана и текущей рабочей версии проекта MTB Galaxy в каталоге D:\MTB. Ниже зафиксированы назначение системы, архитектура, соответствие исходному плану, основные модули, API и порядок запуска приложения.

1. Цель и исходный план

Исходный документ задавал разработку приложения MTB Galaxy как геймифицированного банковского продукта с планетарной картой, социальными механиками, наградами и backend-частью на FastAPI. Базовый план ориентировался на мобильный стек React Native + FastAPI и предусматривал экраны галактики, деталей планет, друзей, AI, QR и профиля.

Текущая рабочая версия проекта реализует ту же бизнес-идею в формате web-MVP: пользователь прокачивает планеты, выполняет квесты, получает награды, запускает мини-игры и взаимодействует с административным контуром. Таким образом, исходный план не просто повторен, а расширен и приведен к демонстрационно-боевому виду.

2. Что реализовано в текущей версии

- Игровой web-клиент на React 19, Vite 6, React Router 7, Zustand, TanStack Query и Framer Motion.
- Backend на FastAPI с SQLAlchemy, Pydantic и локальной SQLite-базой по умолчанию.
- Контракты API в пакете packages/contracts, синхронизированные с OpenAPI.

- Планеты, квесты, награды, лидерборд, реферальная механика и административный контур.
- Реально работающие social/QR/AI модули: маршруты /app/friends, /app/qr, /app/ai и соответствующие backend endpoint-ы /friends, /qr/*, /assistant/*.
- Набор мини-игр: Halva Snake, Credit Shield Reactor, Social Ring Signal и дополнительные игровые страницы MVP.

3. Соответствие исходному плану

Ниже приведено сопоставление основных пунктов исходного Word-плана и фактической реализации в репозитории. Это позволяет показать, какие элементы полностью сохранены и какие были расширены относительно базовой постановки.

Пункт плана	Статус	Фактическая реализация
FastAPI + база данных	Выполнено	Реализован backend на FastAPI; по умолчанию используется SQLite, опционально поддерживаются PostgreSQL и Redis.
Планетарная карта	Выполнено	Сценарий реализован через GalaxyPage и экран карты планет.
Детализация планет/созвездий	Выполнено	Есть отдельный экран планеты с прогрессом, зданиями и играми.
Социальный модуль / друзья	Выполнено	Есть FriendsPage по маршруту /app/friends и backend-методы /friends, /friends/invite, /friends/accept, /friends/activity.
Профиль пользователя	Выполнено	Профиль и сессия доступны через демо-логин, состояние хранится на клиенте и в API.
Игровые механики и подарки	Выполнено с расширением	Добавлены квесты, журнал наград, звездная пыль и несколько мини-игр.
AI и QR экраны из исходного плана	Выполнено	Реализованы QrPage (/app/qr) и AiPage (/app/ai), а backend включает /qr/me, /qr/resolve, /assistant/context и /assistant/chat.

4. Архитектура проекта

Проект построен как монорепозиторий. Пользовательский интерфейс расположен в apps/web, серверная логика и API находятся в apps/api, а типы и контракты вынесены в отдельный пакет packages/contracts. Такой подход

упрощает синхронизацию схем, ускоряет разработку и уменьшает риск расхождения между фронтендом и бэкендом.

```
MTB/
├── apps/
│   ├── api/
│   │   ├── app/
│   │   │   ├── main.py
│   │   │   ├── api.py
│   │   │   ├── db.py
│   │   │   ├── models.py
│   │   │   ├── schemas.py
│   │   │   ├── engine.py
│   │   │   ├── quests.py
│   │   │   ├── analytics.py
│   │   │   └── worker.py
│   │   ├── tests/
│   │   └── openapi.json
│   └── web/
│       ├── src/app/router.tsx
│       ├── src/pages/
│       │   ├── GalaxyPage.tsx
│       │   ├── GamesPage.tsx
│       │   ├── QuestsPage.tsx
│       │   ├── RewardsPage.tsx
│       │   ├── FriendsPage.tsx
│       │   ├── QrPage.tsx
│       │   ├── AiPage.tsx
│       │   ├── ReferralsPage.tsx
│       │   └── Admin*.tsx
│       ├── src/features/planets/screens/
│       └── src/lib/api.ts
├── packages/contracts/src/generated/api.ts
├── scripts/serve-web.mjs
├── docker-compose.yml
└── README.md
```

Корневой package.json содержит сценарии для локальной разработки, одновременного запуска API и web-клиента, генерации контрактов из OpenAPI и запуска тестов. За счет этого проект можно разворачивать как единый стенд для демонстрации и дальнейшей доработки.

5. Backend: FastAPI, модели и движок наград

Серверная часть разработана на FastAPI. Точка входа находится в apps/api/app/main.py: приложение поднимает экземпляр FastAPI, настраивает CORS, создает таблицы и выполняет сидирование демонстрационных данных при старте. Маршруты вынесены в модуль api.py, бизнес-логика прогресса и начислений разделена по модулям engine.py, quests.py, analytics.py и worker.py.

Для хранения данных по умолчанию используется SQLite, однако конфигурация предусматривает расширение до PostgreSQL. Модели и схемы построены на SQLAlchemy и Pydantic. Это позволяет обслуживать как локальный демо-стенд, так и более боевую конфигурацию без полной перестройки архитектуры.

- FastAPI обеспечивает HTTP API и OpenAPI-описание интерфейса.
- SQLAlchemy отвечает за модели, сессии и доступ к данным.
- Pydantic используется для валидации входных и выходных схем.
- Seed-логика создает демо-пользователя и стартовые квесты при запуске.
- Движок наград обрабатывает игровые и банковские события, влияющие на прогресс пользователя.

6. Frontend: web-клиент MTB Galaxy

Клиентская часть написана на React 19.0.0 с использованием Vite 6.2.0, React Router 7.3.0, Zustand и TanStack Query. Маршрутизация реализована в apps/web/src/app/router.tsx: приложение разделено на пользовательский контур и административные страницы.

Главный экран GalaxyPage является центром сценария: на нем показаны планеты, игровые действия, статистика, прогресс и ссылки на остальные разделы. Дополнительно реализованы карты планет, отдельные игровые страницы, квесты, награды, реферальный раздел и админский блок. Интерфейс построен как единое SPA-приложение, что ускоряет демонстрацию и упрощает перенос в разные окружения.

Отдельно от главного контура в web-клиенте присутствуют маршруты /app/friends, /app/qr и /app/ai. Они ведут соответственно на FriendsPage, QrPage и AiPage и закрывают именно те сценарии, которые были заложены в исходном плане: дружеские связи, QR-переходы и AI-помощник.

Ключевой экран/модуль	Назначение
GalaxyPage	Главный экран галактики с прогрессом, планетами и

	действиями пользователя.
PlanetsMapScreen / PlanetDetailScreen	Планетарная карта и детальный просмотр каждой планеты.
GamesPage / GameScreen	Запуск мини-игр и управление доступностью игровых сценариев.
QuestsPage	Отображение квестов и получение наград.
RewardsPage	История наград, звездной пыли и бонусов.
FriendsPage (/app/friends)	Полноценный social MVP: списки друзей, инвайты, входящие заявки и переходы в QR-сценарий.
QrPage (/app/qr)	Экран собственного QR и резолвера payload для маршрутов друзей, рефералов и AI-помощника.
AiPage (/app/ai)	AI-ассистент с контекстом по друзьям, квестам, QR и быстрыми сценариями навигации.
ReferralsPage / LeaderboardScreen	Реферальный сценарий и турнирная таблица игроков.
AdminKpiPage / AdminSimulatorPage / AdminRiskPage	Административный контур: KPI, симулятор событий и риск-мониторинг.

7. Игровая логика и банковые механики

В проекте выделены три основные планеты прогресса, отражающие ключевые банковские сценарии пользователя. Каждая планета имеет собственную логику действий, условия разблокировки и связанную игровую активность. Дополнительно в MVP присутствуют мини-игры и система наград, поддерживающие вовлечение пользователя.

- ORBIT_COMMERCE: сценарии покупок, прогресс планеты и связанные игровые активности.
- CREDIT_SHIELD: дисциплина кредитования, игровые проверки и защитные механики.
- SOCIAL_RING: реферальная логика, приглашения и социальный прогресс.
- Система наград начисляет звездную пыль, бустеры и контейнеры; результаты попадают в журнал наград.

Важной частью реализации является административный контур. Он позволяет демонстрировать продуктовые KPI, моделировать события и отслеживать риск-флаги. За счет этого проект подходит не только как пользовательский прототип, но и как демо-стенд для внутренних команд и презентаций.

8. Основные API-методы

OpenAPI-описание проекта содержит 22 основных endpoint-ов. Они покрывают авторизацию демо-пользователя, профиль галактики, события, квесты, награды, мини-игры, социальный модуль друзей, QR-механику, AI-помощника, рефералов, лидерборд и административный функционал.

Для social/QR/AI контура в репозитории уже опубликованы и используются endpoint-ы /friends, /friends/invite, /friends/accept, /friends/activity, /qr/me, /qr/resolve, /assistant/context и /assistant/chat. Это не заглушки и не адаптационные заметки, а часть текущего рабочего API и маршрутизации web-клиента.

Метод	Путь	Назначение
POST	/auth/demo-login	Создание или обновление демо-сессии пользователя.
GET	/galaxy/profile	Получение профиля галактики, планет, бустеров и прогресса.
POST	/events/ingest	Прием банковского события и запуск движка наград.
GET	/quests	Получение текущих квестов пользователя.
POST	/quests/{quest_id}/claim	Выдача награды за выполненный квест.
GET	/rewards/ledger	Журнал начислений и наград.
POST	/games/runs	Сохранение результата мини-игры.
GET	/games/summary	Сводка по мини-играм и прогрессу.
GET	/friends	Получение списков друзей, входящих и исходящих приглашений.
POST	/friends/invite	Отправка приглашения в друзья вручную, по QR или реферальному сценарию.
POST	/friends/accept	Подтверждение входящего приглашения и фиксация дружеской связи.
GET	/friends/activity	Лента активности друзей и социального контура.
GET	/qr/me	Получение собственного QR payload и CTA для дальнейшего сценария.
POST	/qr/resolve	Разбор QR payload и определение действия: друзья, реферал или AI.
GET	/assistant/context	Сбор контекста AI-помощника по друзьям, квестам и QR-сценариям.
POST	/assistant/chat	Диалог с AI-помощником с

		учетом сообщения пользователя и QR payload.
POST	/referrals/invite	Создание приглашения друга.
GET	/leaderboard	Получение лидерборда игроков.
GET	/admin/kpi	Витрина KPI для административного контура.
POST	/admin/simulate	Симуляция банковского события из админ-панели.
GET	/admin/risk	Просмотр риск-флагов и ожидающих наград.
GET	/admin/stream	SSE-поток обновления административных метрик.

9. Требования и запуск проекта

Для локального запуска необходимо подготовить стандартную среду разработки и установить зависимости как для JavaScript-части, так и для Python API. Базовые требования и рекомендуемый сценарий запуска соответствуют README проекта.

- Node.js 20 или новее.
- Python 3.12 или новее.
- Windows PowerShell, Git Bash или другая оболочка с доступом к npm и Python.

Рекомендуемая последовательность команд для старта стенда:

```
npm install
python -m venv .venv
.venv\Scripts\activate
pip install -e apps/api[dev]
npm run contracts:generate
npm run dev
```

После запуска приложение открывается по адресу http://localhost:5173, а API работает на порту 8000. Для синхронизации контракта фронтенда с FastAPI используется отдельная команда генерации OpenAPI-типов.

10. Проверка качества

Для проверки целостности проекта предусмотрены автоматические команды тестирования и сборки. Их следует выполнять перед демонстрацией, публикацией или передачей проекта. Набор проверок покрывает backend, frontend, сборку web-клиента и генерацию контрактов.

```
npm run test:api
npm run test:web
npm run build:web
npm run contracts:generate
```

Дополнительно рекомендуется ручной прогон ключевых экранов на нескольких ширинах экрана, а также проверка игровых сценариев, квестов, лидерборда и административной панели. Такой порядок позволяет быстро убедиться, что проект соответствует ожиданиям исходного плана и корректно показывает расширенный функционал MVP.

11. Вывод

По итогам анализа можно сделать вывод, что MTB Galaxy реализован как полноценный демонстрационный продукт на базе FastAPI и современного React web-стека. По отношению к исходному Word-плану проект не только сохраняет ключевые идеи планетарной карты, прогресса, наград и социального сценария, но и расширяет их за счет квестов, административного контура, контрактов API и нескольких мини-игр.

Итоговая версия документа подходит как для сдачи, внутренней презентации или сопровождения проекта: она фиксирует архитектуру, текущее состояние реализации и порядок запуска без необходимости показывать сырой набор файлов и черновые фрагменты кода из исходного плана.